

Construindo um Servidor de Pedidos Concorrente em Go

Um exemplo completo utilizando Mutex, WaitGroup, defer e sync.Once

Um dos maiores desafios da programação concorrente é permitir que diversas tarefas aconteçam ao mesmo tempo sem que uma interfira na outra. Em Go, isso acontece através das **goroutines**, que são extremamente leves e permitem executar centenas ou milhares de tarefas simultaneamente. Entretanto, quando essas goroutines compartilham os mesmos dados, surgem problemas conhecidos como **race conditions**, onde duas ou mais goroutines modificam a mesma variável ao mesmo tempo, produzindo resultados imprevisíveis. A aula apresenta justamente como evitar esses problemas utilizando os mecanismos de sincronização do pacote `sync`, como **Mutex**, **WaitGroup** e **sync.Once**.

Neste artigo construiremos um exemplo inspirado em um sistema real: um servidor de e-commerce que recebe diversos pedidos simultaneamente. O sistema precisa inicializar o banco de dados apenas uma vez, processar vários pedidos em paralelo, atualizar um contador global de pedidos de maneira segura e somente encerrar quando todos os pedidos forem concluídos.

Cenário

Imagine uma loja virtual recebendo centenas de pedidos ao mesmo tempo.

Cada pedido será executado por uma goroutine diferente.

Todos os pedidos utilizam:

- uma única conexão com o banco;
- um contador global de pedidos;
- processamento paralelo.

Sem sincronização, o contador poderá ficar incorreto e o banco poderá ser inicializado várias vezes.

O código completo

```
package main

import (
    "fmt"
    "sync"
    "time"
)

////////////////////////////////////
// Contador global de pedidos
////////////////////////////////////

var totalPedidos int

////////////////////////////////////
// Mutex protege o contador compartilhado
```

```

////////////////////////////////////
var mu sync.Mutex

////////////////////////////////////
// Once garante que o banco será iniciado apenas uma vez
////////////////////////////////////

var once sync.Once

////////////////////////////////////
// Simula inicialização do banco
////////////////////////////////////

func inicializarBanco() {

    fmt.Println("Conectando ao banco de dados...")

    time.Sleep(2 * time.Second)

    fmt.Println("Banco conectado com sucesso!")
}

////////////////////////////////////
// Processa um pedido
////////////////////////////////////

func processarPedido(cliente int, wg *sync.WaitGroup) {

    // Informa ao WaitGroup que a tarefa terminou
    defer wg.Done()

    //////////////////////////////////////
    // Inicializa o banco apenas uma única vez
    //////////////////////////////////////

    once.Do(inicializarBanco)

    fmt.Println("Cliente", cliente, "enviou um pedido.")

    //////////////////////////////////////
    // Simula tempo de processamento
    //////////////////////////////////////

    time.Sleep(time.Second)

    //////////////////////////////////////
    // Região crítica
    //////////////////////////////////////

    mu.Lock()

    defer mu.Unlock()

    totalPedidos++

    fmt.Println("Pedido do cliente", cliente,
        "processado. Total:", totalPedidos)
}

////////////////////////////////////

func main() {

```

```

var wg sync.WaitGroup

////////////////////////////////////////
// Simula 20 clientes acessando juntos
////////////////////////////////////////

for i := 1; i <= 20; i++ {

    wg.Add(1)

    go processarPedido(i, &wg)

}

////////////////////////////////////////
// Espera todos terminarem
////////////////////////////////////////

wg.Wait()

fmt.Println()
fmt.Println("=====")
fmt.Println("Todos os pedidos foram concluídos")
fmt.Println("Quantidade final:", totalPedidos)
}

```

Entendendo o programa

Embora o código pareça grande, ele é composto por quatro mecanismos principais, cada um responsável por resolver um problema diferente.

1. WaitGroup

O primeiro componente utilizado é o **WaitGroup**.

```
var wg sync.WaitGroup
```

Ele funciona como um contador de tarefas.

Sempre que criamos uma nova goroutine fazemos:

```
wg.Add(1)
```

Quando ela termina:

```
defer wg.Done()
```

E no final do programa:

```
wg.Wait()
```

O programa principal fica parado esperando todas as goroutines terminarem.

Sem isso, o programa poderia encerrar antes mesmo dos pedidos serem processados.

2. sync.Once

Observe este trecho:

```
once.Do(inicializarBanco)
```

Imagine que vinte clientes chegaram exatamente no mesmo instante.

Todos precisam acessar o banco.

Sem o `sync.Once`, cada goroutine poderia tentar conectar ao banco novamente.

Teríamos algo parecido com:

```
Conectando...
Conectando...
Conectando...
Conectando...
Conectando...
```

Isso desperdiçaria memória e processamento.

O `sync.Once` resolve esse problema.

Mesmo que vinte goroutines chamem:

```
once.Do(inicializarBanco)
```

A função será executada apenas uma única vez.

Todas as outras aguardam a conclusão da primeira.

Esse padrão é conhecido como **Singleton**.

3. Mutex

Agora chegamos ao ponto mais importante.

Todos os pedidos precisam aumentar o contador global:

```
totalPedidos++
```

Parece uma única instrução.

Mas internamente ela ocorre em três etapas:

```
ler valor
```

```
somar 1
```

```
gravar valor
```

Imagine duas goroutines acontecendo ao mesmo tempo.

As duas leem:

```
10
```

As duas somam:

```
11
```

As duas gravam:

11

Quando o correto seria:

12

Esse é exatamente o problema chamado **Race Condition**.

Para impedir isso utilizamos:

```
mu.Lock()
```

Somente uma goroutine entra na região crítica.

Todas as outras ficam aguardando.

Quando termina:

```
mu.Unlock()
```

A próxima goroutine pode continuar.

4. defer

Logo após fazer o Lock encontramos:

```
defer mu.Unlock()
```

Esse é um dos padrões mais utilizados em Go.

Mesmo que aconteça:

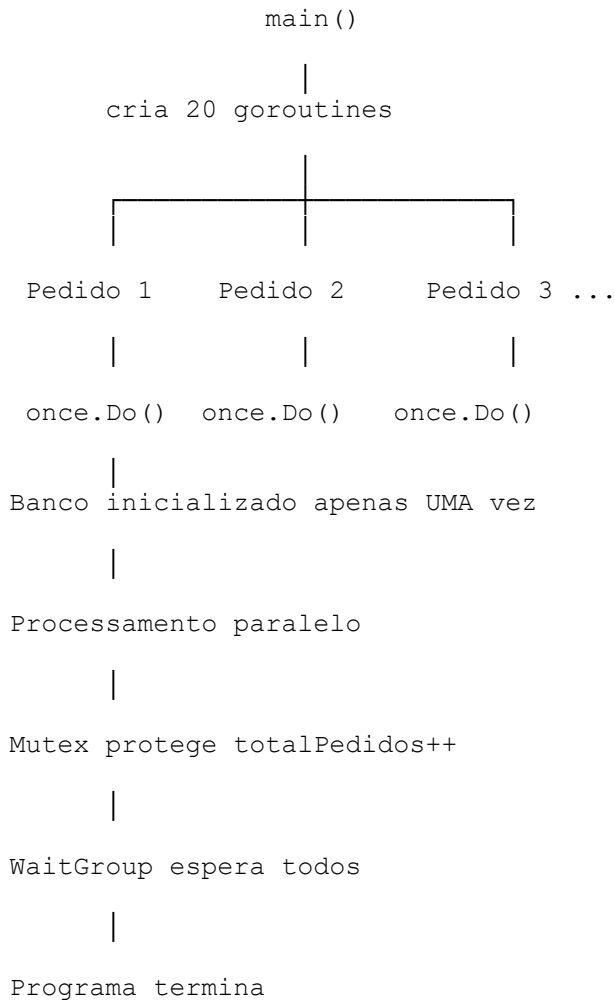
- um return;
- um erro;
- um panic;

o Unlock sempre será executado.

Isso evita um dos erros mais comuns em concorrência: esquecer de liberar o Mutex e causar um **deadlock**, situação em que outras goroutines ficam bloqueadas indefinidamente.

Fluxo completo do programa

Quando executamos o programa, acontece algo semelhante a isto:



Observe que existem duas sincronizações acontecendo ao mesmo tempo.

O **WaitGroup** controla quando o programa termina.

O **Mutex** controla quem pode alterar o contador.

Enquanto isso, o **sync.Once** garante que a inicialização do banco aconteça apenas uma vez.

Cada ferramenta resolve um problema diferente.

O que aconteceria sem essas proteções?

Sem **WaitGroup**:

- o programa encerraria antes das goroutines.

Sem **Mutex**:

- o contador terminaria com um valor incorreto.

Sem **sync.Once**:

- cada cliente tentaria abrir uma nova conexão com o banco.

Sem **defer**:

- seria fácil esquecer de liberar o Mutex e travar o programa.

Onde esse padrão é utilizado?

Este modelo aparece diariamente em aplicações profissionais, como:

- APIs REST que recebem centenas de requisições simultâneas.
- Servidores Web escritos em Go.
- Sistemas bancários processando transações.
- Plataformas de e-commerce.
- Sistemas de filas (RabbitMQ, Kafka, NATS).
- Processamento paralelo de arquivos.
- Microserviços que compartilham cache ou configurações globais.

Conclusão

Este exemplo reúne praticamente todos os conceitos centrais da aula: o **Mutex** protege dados compartilhados contra acessos simultâneos, o **WaitGroup** garante que todas as goroutines terminem antes do encerramento do programa, o **sync.Once** implementa uma inicialização única (Singleton) e o uso de **defer** assegura que recursos críticos sejam sempre liberados corretamente. Em conjunto, esses mecanismos permitem criar aplicações concorrentes seguras, organizadas e escaláveis, exatamente como acontece em servidores web, APIs e sistemas corporativos desenvolvidos em Go.